

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Мегафакультет компьютерных технологий и управления

Факультет программной инженерии и компьютерной техники

Лабораторная работа №4
по предмету “базы данных”

Выполнила: Кобленц Мария Алексеевна

Группа: Р3106

Проверил: Мартин Райла

г. Санкт-Петербург

2026

Задание.....	3
Запрос 1.....	4
Реализация запроса.....	4
Планы выполнения.....	4
На гелиосе EXPLAIN ANALYZE.....	6
Выполнение в своей среде без индексов.....	6
Индексы.....	6
Выполнение после добавки индексов.....	7
Запрос 2.....	8
Реализация запроса.....	8
Планы выполнения.....	9
На гелиосе EXPLAIN ANALYZE.....	10
Выполнение в своей среде без индексов.....	10
Индексы.....	11
После добавления индексов.....	12
Задание на паре.....	13

Задание.

Введите вариант:

Внимание! У разных вариантов разный текст задания!

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_СЕССИЯ.УЧГОД.

Фильтры (AND):

a) Н_ЛЮДИ.ИМЯ > Александр.

b) Н_СЕССИЯ.УЧГОД > 2011/2012.

c) Н_СЕССИЯ.УЧГОД > 2008/2009.

Вид соединения: LEFT JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.НЗК, Н_УЧЕНИКИ.ГРУППА.

Фильтры: (AND)

a) Н_ЛЮДИ.ИД > 152862.

b) Н_ОБУЧЕНИЯ.НЗК = 999080.

Вид соединения: RIGHT JOIN.



Запрос 1.

Реализация запроса

Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_СЕССИЯ.УЧГОД.

Фильтры (AND):

а) Н_ЛЮДИ.ИМЯ > Александр.

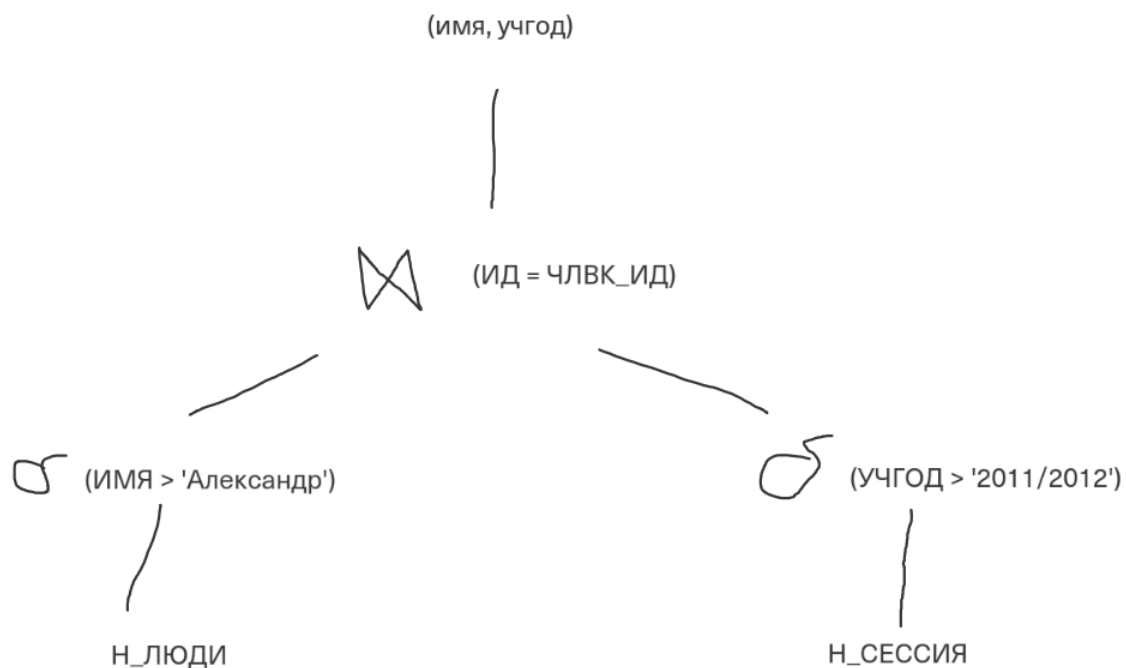
б) Н_СЕССИЯ.УЧГОД > 2011/2012.

с) Н_СЕССИЯ.УЧГОД > 2008/2009.

Вид соединения: LEFT JOIN.

Условие с избыточно.

```
select л."ИМЯ",  
       нс."УЧГОД"  
from "Н_ЛЮДИ" л left join public."Н_СЕССИЯ" нс on л."ИД" = нс."ЧЛВК_ИД"  
where л."ИМЯ">'Александр' and нс."УЧГОД">'2011/2012' and нс."УЧГОД">'2008/2009';
```



Планы выполнения.

План 1.

Nested Loop Join

- > Seq Scan on H_ЛЮДИ л
Filter: (ИМЯ > 'Александр')
- > Seq Scan on H_СЕССИЯ нс
Filter: (УЧГОД > '2011/2012' AND ЧЛВК_ИД = л.ИД)

Почему возможен: Для малых таблиц PostgreSQL может выбрать вложенные циклы вместо хеш-соединения.

План 2.

Merge Join

- Merge Cond: (л.ИД = нс.ЧЛВК_ИД)
- > Sort
 - > Seq Scan on H_ЛЮДИ л
Filter: (ИМЯ > 'Александр')
- > Sort
 - > Seq Scan on H_СЕССИЯ нс
Filter: (УЧГОД > '2011/2012')

Почему возможен: Если оптимизатор решит, что сортировка + слияние эффективнее.

План 3. Наиболее логичный и вероятный план. С ним буду сравнивать.

Hash Left Join

- Hash Cond: (л.ИД = нс.ЧЛВК_ИД)
- > Seq Scan on H_ЛЮДИ л
Filter: (ИМЯ > 'Александр'::text)
- > Hash
 - > Seq Scan on H_СЕССИЯ нс
Filter: (УЧГОД > '2011/2012'::text AND УЧГОД > '2008/2009'::text)

Почему выбран этот план:

Seq Scan читает все страницы таблиц. Фильтрация применяется "на лету". Hash Join строит хеш-таблицу по результатам сканирования H_СЕССИЯ (правая таблица в LEFT JOIN) и проходит по H_ЛЮДИ. Это оптимально, так как позволяет избежать сложности $O(N \times M)$.

На гелиосе EXPLAIN ANALYZE

QUERY PLAN	
1	Nested Loop (cost=0.28..135.59 rows=1 width=23) (actual time=1.407..1.408 rows=0 loops=1)
2	-> Seq Scan on "Н_СЕССИЯ" "нс" (cost=0.00..127.28 rows=1 width=14) (actual time=1.406..1.406 rows=0 loops=1)
3	Filter: (((("УЧГОД")::text > '2011/2012')::text) AND ((("УЧГОД")::text > '2008/2009')::text))
4	Rows Removed by Filter: 3752
5	-> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" "л" (cost=0.28..8.30 rows=1 width=17) (never executed)
6	Index Cond: ("ИД" = "нс"."ЧЛВК_ИД")
7	Filter: (("ИМЯ")::text > 'Александр')::text
8	Planning Time: 0.400 ms
9	Execution Time: 1.444 ms

Выполнение в своей среде без индексов

Так как запрос мне ничего не вернул, а у нас задание создать среду, в которой будут найдены подходящие данные, я смогла проверить как будет работать этот запрос без индексов.

QUERY PLAN	
Hash Join (cost=11.00..25.14 rows=8 width=84) (actual time=3.038..3.043 rows=2.00 loops=1)	
Hash Cond: ("нс"."ЧЛВК_ИД" = "л"."ИД")	
Buffers: shared hit=2 dirtied=1	
-> Seq Scan on "Н_СЕССИЯ" "нс" (cost=0.00..13.75 rows=83 width=40) (actual time=0.031..0.034 rows=4.00 loops=1)	
Filter: (((("УЧГОД")::text > '2011/2012')::text) AND ((("УЧГОД")::text > '2008/2009')::text))	
Rows Removed by Filter: 1	
Buffers: shared hit=1	
-> Hash (cost=10.75..10.75 rows=20 width=52) (actual time=2.991..2.991 rows=4.00 loops=1)	
Buckets: 1024 Batches: 1 Memory Usage: 9kB	
Buffers: shared hit=1 dirtied=1	
-> Seq Scan on "Н_ЛЮДИ" "л" (cost=0.00..10.75 rows=20 width=52) (actual time=2.982..2.986 rows=4.00 loops=1)	
Filter: (("ИМЯ")::text > 'Александр')::text	
Rows Removed by Filter: 1	
Buffers: shared hit=1 dirtied=1	
Planning Time: 0.099 ms	
Execution Time: 3.067 ms	
(16 rows)	

Предполагалось: Hash Left Join

Реальность: Hash Join (внутреннее соединение)

Объяснение:

PostgreSQL выполнил оптимизацию запроса, преобразовав LEFT JOIN в INNER JOIN. Это произошло потому, что условие фильтрации по правой таблице (нс."УЧГОД" > '2011/2012') находится в блоке WHERE, а не в блоке ON. Оптимизатор понимает, что строки из левой таблицы, не нашедшие соответствия в правой (где нс."УЧГОД" будет NULL), всё равно будут отфильтрованы условием WHERE, так как сравнение NULL > '2011/2012' возвращает UNKNOWN (ложь). Поэтому нет смысла сохранять такие строки, и LEFT JOIN заменяется на более эффективный INNER JOIN.

Индексы

В PostgreSQL оптимальным типом для условий >, <, = и соединений по равенству является B-Tree.

Индекс 1

Таблица: Н_ЛЮДИ

Атрибут: ИМЯ

Тип индекса: B-Tree

Обоснование: Фильтр ИМЯ > 'Александр' требует быстрого поиска по диапазону строк.

Индекс 2

Таблица: Н_СЕССИЯ

Атрибут: УЧГОД

Тип индекса: B-Tree

Обоснование: Фильтр УЧГОД > '2011/2012' также работает по диапазону. Индекс ускорит отсев строк до соединения.

Выполнение после добавки индексов

```
CREATE INDEX idx_lyudi_imya ON "Н_ЛЮДИ"("ИМЯ");  
CREATE INDEX idx_sessiya_uchgod ON "Н_СЕССИЯ"("УЧГОД");
```

```
merge л: имя > Александр and нс: учгод > 2011/2012 and нс: учгод > 2008/2009 ;  
QUERY PLAN  
-----  
Merge Join (cost=968.10..1555.92 rows=7513 width=17) (actual time=5.043..9.090 rows=6536.00 loops=1)  
  Merge Cond: ("л"."ИД" = "нс"."ЧЛБК_ИД")  
  Buffers: shared hit=446 read=24  
    -> Index Scan using idx_lyudi_id on "Н_ЛЮДИ" "л" (cost=0.29..4855.03 rows=99011 width=11) (actual time=0.008..2.400 rows=9016.00 loops=1)  
        Filter: (("ИМЯ")::text > 'Александр'::text)  
        Rows Removed by Filter: 3  
        Index Searches: 1  
        Buffers: shared hit=187 read=24  
    -> Sort (cost=967.81..986.59 rows=7512 width=14) (actual time=5.030..5.312 rows=7534.00 loops=1)  
        Sort Key: "нс"."ЧЛБК_ИД"  
        Sort Method: quicksort Memory: 427kB  
        Buffers: shared hit=259  
        -> Seq Scan on "Н_СЕССИЯ" "нс" (cost=0.00..484.23 rows=7512 width=14) (actual time=0.007..3.760 rows=7512.00 loops=1)  
            Filter: (((("УЧГОД")::text > '2011/2012'::text) AND ((("УЧГОД")::text > '2008/2009'::text)))  
            Rows Removed by Filter: 7503  
            Buffers: shared hit=259  
Planning:  
  Buffers: shared hit=35  
Planning Time: 0.308 ms  
Execution Time: 9.330 ms  
(20 rows)
```

ЧТО ИЗМЕНИЛОСЬ И ПОЧЕМУ:

1. Появился Index Scan вместо Seq Scan для таблицы Н_ЛЮДИ
Причина: Индекс по полю ИД позволяет быстро находить строки без полного перебора таблицы. Вместо чтения всех 10000 строк оптимизатор использует индексное дерево.
2. Изменился тип соединения с Hash Join на Merge Join
Причина: Благодаря индексу таблица Н_ЛЮДИ теперь поступает в соединение уже отсортированной. Оптимизатору выгоднее отсортировать вторую таблицу (Н_СЕССИЯ) и выполнить слияние (Merge), чем строить хеш-таблицу.

Запрос 2

Реализация запроса

Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.НЗК, Н_УЧЕНИКИ.ГРУППА.

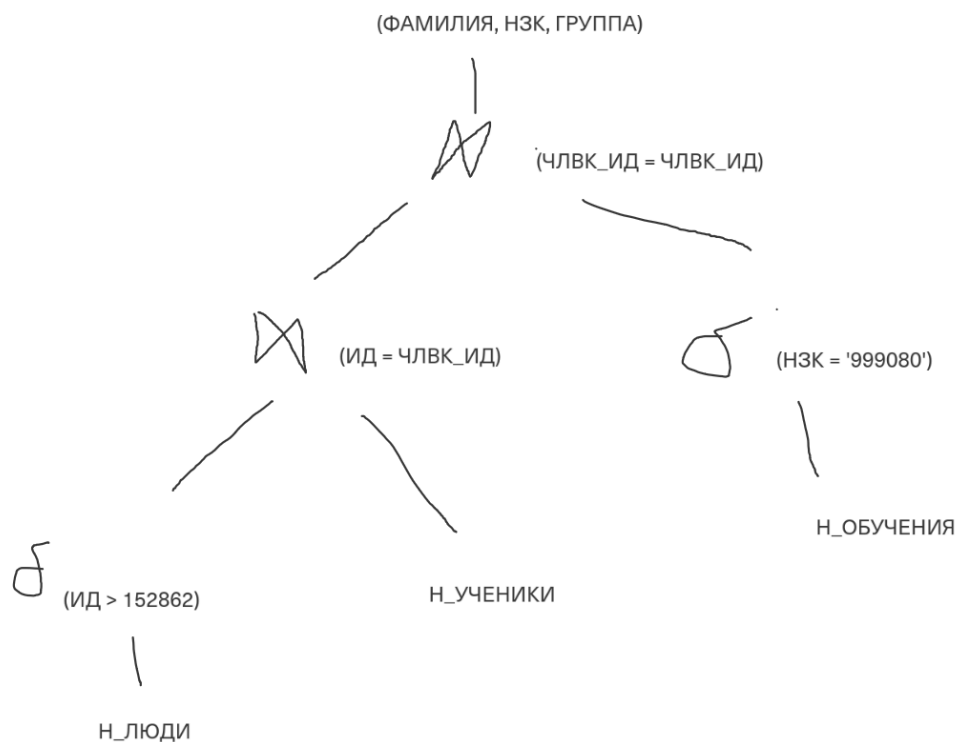
Фильтры: (AND)

а) Н_ЛЮДИ.ИД > 152862.

б) Н_ОБУЧЕНИЯ.НЗК = 999080.

Вид соединения: RIGHT JOIN.

```
select л."ФАМИЛИЯ",  
       о."НЗК",  
       у."ГРУППА"  
from "Н_ЛЮДИ" л right join "Н_УЧЕНИКИ" у on л."ИД" = у."ЧЛВК_ИД"  
      right join "Н_ОБУЧЕНИЯ" о on о."ЧЛВК_ИД" = у."ЧЛВК_ИД"  
where л."ИД" > 152862  
and о."НЗК" like '999080';
```



Планы выполнения

План 1.

Nested Loop

-> Seq Scan on H_ОБУЧЕНИЯ o

Filter: (H3K = '999080')

-> Nested Loop

-> Seq Scan on H_УЧЕНИКИ y

Filter: (ЧЛВК_ИД = o.ЧЛВК_ИД)

-> Seq Scan on H_ЛЮДИ л

Filter: (ИД > 152862 AND ИД = y.ЧЛВК_ИД)

Почему возможен: При малом количестве строк после фильтрации.

План 2.

Hash Join

Hash Cond: (y.ЧЛВК_ИД = л.ИД)

-> Hash Join

Hash Cond: (o.ЧЛВК_ИД = y.ЧЛВК_ИД)

-> Seq Scan on H_ОБУЧЕНИЯ o

Filter: (H3K = '999080')

-> Seq Scan on H_УЧЕНИКИ y

-> Seq Scan on H_ЛЮДИ л

Filter: (ИД > 152862)

Почему возможен: Другой порядок соединения таблиц.

План 3. Наиболее логичный и вероятный план. С ним буду сравнивать.

Hash Right Join

Hash Cond: (o.ЧЛВК_ИД = y.ЧЛВК_ИД)

-> Seq Scan on H_ОБУЧЕНИЯ o

Filter: (H3K ~~ '999080'::text)

-> Hash

Hash Cond: (y.ЧЛВК_ИД = л.ИД)

-> Seq Scan on H_УЧЕНИКИ y

-> Hash

-> Seq Scan on H_ЛЮДИ л

Filter: (ИД > 152862)

Почему выбран этот план:

Оптимизатор переписывает цепочку RIGHT JOIN во внутреннее представление.

Сначала фильтруются таблицы по условиям WHERE, затем данные объединяются через Hash Join. Сортировка не требуется, что экономит ресурсы CPU.

На гелиосе EXPLAIN ANALYZE

QUERY PLAN	
1	Nested Loop (cost=0.57..131.49 rows=1 width=26) (actual time=0.776..0.777 rows=0 loops=1)
2	-> Nested Loop (cost=0.28..128.08 rows=1 width=30) (actual time=0.776..0.776 rows=0 loops=1)
3	-> Seq Scan on "Н_ОБУЧЕНИЯ" "о" (cost=0.00..119.76 rows=1 width=10) (actual time=0.164..0.771 rows=1 loops=1)
4	Filter: (("НЗК")::text ~~ '999080')::text)
5	Rows Removed by Filter: 5020
6	-> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" "н" (cost=0.28..8.30 rows=1 width=20) (actual time=0.003..0.003 rows=0 loops=1)
7	Index Cond: (("ИД" = "о"."ЧЛВК_ИД") AND ("ИД" > 152862))
8	-> Index Scan using "УЧЕН_ОБУЧ_ФК_I" on "Н_УЧЕНИКИ" "у" (cost=0.29..3.36 rows=5 width=8) (never executed)
9	Index Cond: ("ЧЛВК_ИД" = "н"."ИД")
10	Planning Time: 0.616 ms
11	Execution Time: 0.818 ms

Выполнение в своей среде без индексов

Так как запрос мне ничего не вернул, а у нас задание создать среду, в которой будут найдены подходящие данные, я смогла проверить как будет работать этот запрос без индексов.

QUERY PLAN	

Nested Loop (cost=13.51..135.35 rows=1 width=122) (actual time=0.096..0.114 rows=1.00 loops=1)	
Join Filter: ("н"."ИД" = "у"."ЧЛВК_ИД")	
Rows Removed by Join Filter: 5	
Buffers: shared hit=5	
-> Hash Join (cost=13.51..24.35 rows=1 width=62) (actual time=0.042..0.048 rows=3.00 loops=1)	
Hash Cond: ("у"."ЧЛВК_ИД" = "о"."ЧЛВК_ИД")	
Buffers: shared hit=2	
-> Seq Scan on "Н_УЧЕНИКИ" "у" (cost=0.00..10.60 rows=60 width=24) (actual time=0.017..0.018 rows=4.00 loops=1)	
Buffers: shared hit=1	
-> Hash (cost=13.50..13.50 rows=1 width=38) (actual time=0.018..0.019 rows=3.00 loops=1)	
Buckets: 1024 Batches: 1 Memory Usage: 9kB	
Buffers: shared hit=1	
-> Seq Scan on "Н_ОБУЧЕНИЯ" "о" (cost=0.00..13.50 rows=1 width=38) (actual time=0.010..0.013 rows=3.00 loops=1)	
Filter: (("НЗК")::text ~~ '999080')::text)	
Rows Removed by Filter: 1	
Buffers: shared hit=1	
-> Seq Scan on "Н_ЛЮДИ" "н" (cost=0.00..10.75 rows=20 width=72) (actual time=0.018..0.019 rows=2.00 loops=3)	
Filter: ("ИД" > 152862)	
Rows Removed by Filter: 11	
Buffers: shared hit=3	
Planning Time: 0.283 ms	
Execution Time: 0.156 ms	
(22 rows)	

Сравнение:

Тип соединения на верхнем уровне:

Предполагалось: Hash Right Join

Реальность: Nested Loop

Объяснение:

PostgreSQL оптимизатор переписал RIGHT JOIN во внутреннее представление, где порядок таблиц изменён. Вместо одного Hash Join для всех трёх таблиц, оптимизатор выбрал двухэтапное соединение:

Сначала Hash Join между Н_УЧЕНИКИ и Н_ОБУЧЕНИЯ

Затем Nested Loop с таблицей Н_ЛЮДИ

Такое решение принято потому, что после первого Hash Join осталось всего 3 строки, и для каждой из них эффективнее выполнить простое последовательное сканирование Н_ЛЮДИ (Nested Loop), чем строить ещё одну хеш-таблицу.

Порядок соединения таблиц:

Предполагалось: Н_ОБУЧЕНИЯ → Н_УЧЕНИКИ → Н_ЛЮДИ

Реальность: Н_УЧЕНИКИ → Н_ОБУЧЕНИЯ → Н_ЛЮДИ

Объяснение:

Seq Scan on Н_УЧЕНИКИ возвращает 4 строки

Seq Scan on Н_ОБУЧЕНИЯ с фильтром возвращает 3 строки

Hash Join между ними даёт 3 строки

Затем для каждой из 3 строк выполняется Seq Scan on Н_ЛЮДИ (4 строки после фильтра)

Индексы

Индекс 1.

Таблица: Н_ЛЮДИ

Атрибут: ИД

Тип индекса: B-Tree

Обоснование: Условие ИД > 152862 обращается к первичному ключу. Явный индекс ускорит Range Scan.

Индекс 2.

Таблица: Н_ОБУЧЕНИЯ

Атрибут: НЗК

Тип индекса: B-Tree

Обоснование: Условие НЗК LIKE '999080' эквивалентно точному совпадению. Индекс позволит найти строку за $O(\log N)$.

Индекс 3.

Таблицы: Н_СЕССИЯ, Н_УЧЕНИКИ, Н_ОБУЧЕНИЯ

Атрибут: ЧЛВК_ИД

Тип индекса: B-Tree

Обоснование: Поля используются в условиях JOIN. Индексы на внешние ключи ускоряют поиск совпадений при соединении таблиц.

После добавления индексов

```
CREATE INDEX idx_lyudi_id ON "Н_ЛЮДИ"("ИД");
CREATE INDEX idx_obucheniya_nzk ON "Н_ОБУЧЕНИЯ"("НЗК");
CREATE INDEX idx_sessiya_chlvk_id ON "Н_СЕССИЯ"("ЧЛВК_ИД");
CREATE INDEX idx_ucheniki_chlvk_id ON "Н_УЧЕНИКИ"("ЧЛВК_ИД");
CREATE INDEX idx_obucheniya_chlvk_id ON "Н_ОБУЧЕНИЯ"("ЧЛВК_ИД");
```

```
QUERY PLAN
-----
Nested Loop (cost=4.86..28.16 rows=1 width=55) (actual time=0.135..2.135 rows=2.00 loops=1)
  Join Filter: ("л"."ИД" = "о"."ЧЛВК_ИД")
  Rows Removed by Join Filter: 9610
  Buffers: shared hit=479
  -> Nested Loop (cost=0.57..16.62 rows=1 width=29) (actual time=0.013..0.024 rows=4.00 loops=1)
    Buffers: shared hit=15
    -> Index Scan using idx_lyudi_id on "Н_ЛЮДИ" "л" (cost=0.29..8.31 rows=1 width=18) (actual time=0.004..0.006 rows=4.00 loops=1)
      Index Cond: ("ИД" > 152862)
      Index Searches: 1
      Buffers: shared hit=3
    -> Index Scan using idx_ucheniki_chlvk_id on "Н_УЧЕНИКИ" "у" (cost=0.28..8.30 rows=1 width=11) (actual time=0.002..0.002 rows=1.00 loops=4)
      Index Cond: ("ЧЛВК_ИД" = "л"."ИД")
      Index Searches: 4
      Buffers: shared hit=12
  -> Bitmap Heap Scan on "Н_ОБУЧЕНИЯ" "о" (cost=4.29..11.51 rows=2 width=38) (actual time=0.057..0.380 rows=2403.00 loops=4)
    Filter: (("НЗК")::text ~ '999080')::text)
    Heap Blocks: exact=452
    Buffers: shared hit=464
    -> Bitmap Index Scan on idx_obucheniya_nzk (cost=0.00..4.29 rows=2 width=0) (actual time=0.043..0.043 rows=2403.00 loops=4)
      Index Cond: (("НЗК")::text = '999080')::text)
      Index Searches: 4
      Buffers: shared hit=12
```

ЧТО ИЗМЕНИЛОСЬ И ПОЧЕМУ:

1. Появился Index Scan на таблице Н_ЛЮДИ
Причина: Условие ИД > 152862 очень селективно - оно выбирает только 4 строки из 10000. Индекс позволяет найти эти строки мгновенно, не сканируя всю таблицу.
2. Появился Index Scan на таблице Н_УЧЕНИКИ
Причина: Для каждой из 4 найденных записей Н_ЛЮДИ оптимизатор использует индекс по ЧЛВК_ИД для быстрого поиска соответствующих учеников. Это работает как точечный поиск вместо полного перебора.
3. Появился Bitmap Index Scan на таблице Н_ОБУЧЕНИЯ
Причина: Условие НЗК = '999080' выбирает около 2400 строк из 12000 (каждую 5-ю). Это слишком много для обычного Index Scan, но Bitmap Scan эффективно справляется:
 - Сначала строится битовая карта блоков, содержащих нужные данные
 - Затем эти блоки читаются последовательно. Это быстрее, чем случайные чтения при обычном Index Scan.
4. Hash Join исчез, остался только Nested Loop
Причина: После применения индексов количество строк на каждом этапе резко сократилось. Для малого количества строк вложенные циклы (Nested Loop) эффективнее хеш-соединений.

Задание на паре

Код для создания таблиц

```
CREATE TABLE users (  
    id          serial PRIMARY KEY,  
    username text NOT NULL  
);  
  
CREATE TABLE tasks (  
    id          serial PRIMARY KEY,  
    user_id     int REFERENCES users(id),  
    priority_range int4range NOT NULL  
);  
  
CREATE TABLE task_logs (  
    id          serial PRIMARY KEY,  
    task_id     int REFERENCES tasks(id),  
    status      text NOT NULL  
);
```

Код для генерации данных

6

```
BEGIN;  
  
INSERT INTO users (username)  
SELECT username 'user_' || i FROM generate_series(1, 100000) AS i;  
  
INSERT INTO tasks (user_id, priority_range)  
SELECT  
    floor(random() * 100000 + 1)::int,  
    priority_range int4range(floor(random() * 60)::int, floor(random() * 40)::int + 60)  
FROM generate_series(1, 500000);  
  
INSERT INTO task_logs (task_id, status)  
SELECT  
    floor(random() * 500000 + 1)::int,  
    status (ARRAY['pending', 'active', 'completed', 'failed'])[floor(random() * 4 + 1)]  
FROM generate_series(1, 1500000);  
  
COMMIT;
```

Запрос:

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT u.username, t.priority_range, l.status
FROM users u
      JOIN tasks t  1<->1..n: ON t.user_id = u.id
      JOIN task_logs l  1<->1..n: ON l.task_id = t.id
WHERE u.username = 'user_142'
      AND t.priority_range && int4range(20, 80)
      AND l.status = 'active';

```

Индексы, которые создам

```

CREATE INDEX idx_users_username ON users USING btree (username);
CREATE INDEX idx_tasks_priority ON tasks USING gist (priority_range);
CREATE INDEX idx_logs_status ON task_logs USING btree (status);

```

До добавления индексов:

QUERY PLAN	
1	Gather (cost=9127.10..25997.90 rows=4 width=32) (actual time=77.808..114.965 rows=8.00 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	Buffers: shared hit=13328
5	-> Parallel Hash Join (cost=8127.10..24997.50 rows=2 width=32) (actual time=50.997..81.045 rows=2.67 loops=3)
6	Hash Cond: (l.task_id = t.id)
7	Buffers: shared hit=13328
8	-> Parallel Seq Scan on task_logs l (cost=0.00..16282.50 rows=156771 width=12) (actual time=0.067..39.365 rows=124673.33 loops=3)
9	Filter: (status = 'active'::text)
10	Rows Removed by Filter: 375327
11	Buffers: shared hit=8470
12	-> Parallel Hash (cost=8127.08..8127.08 rows=2 width=28) (actual time=36.426..36.434 rows=2.33 loops=3)
13	Buckets: 1024 Batches: 1 Memory Usage: 104kB
14	Buffers: shared hit=4858
15	-> Hash Join (cost=1791.01..8127.08 rows=2 width=28) (actual time=12.075..36.298 rows=2.33 loops=3)
16	Hash Cond: (t.user_id = u.id)
17	Buffers: shared hit=4858
18	-> Parallel Seq Scan on tasks t (cost=0.00..5789.17 rows=208333 width=22) (actual time=0.097..24.314 rows=166666.67 loops=3)
19	Filter: (priority_range && '[20,80)'::int4range)
20	Buffers: shared hit=3235
21	-> Hash (cost=1791.00..1791.00 rows=1 width=14) (actual time=5.391..5.393 rows=1.00 loops=3)
22	Buckets: 1024 Batches: 1 Memory Usage: 9kB
23	Buffers: shared hit=1623
24	-> Seq Scan on users u (cost=0.00..1791.00 rows=1 width=14) (actual time=0.019..5.383 rows=1.00 loops=3)
25	Filter: (username = 'user_142'::text)
26	Rows Removed by Filter: 99999
27	Buffers: shared hit=1623
28	Planning:

После индексов

Gather (cost=11544.90..22562.84 rows=4 width=32) (actual time=94.982..119.850
 rows=8.00 loops=1)
 Workers Planned: 2
 Workers Launched: 2
 Buffers: shared hit=11717 read=320
 -> Parallel Hash Join (cost=10544.90..21562.44 rows=2 width=32) (actual
 time=65.512..82.548 rows=2.67 loops=3)
 Hash Cond: (l.task_id = t.id)
 Buffers: shared hit=11717 read=320
 -> Parallel Bitmap Heap Scan on task_logs l (cost=4200.36..14630.00 rows=156771
 width=12) (actual time=12.616..46.334 rows=124673.33 loops=3)
 Recheck Cond: (status = 'active'::text)
 Heap Blocks: exact=3247
 Buffers: shared hit=8471 read=317
 Worker 0: Heap Blocks: exact=2642
 Worker 1: Heap Blocks: exact=2581
 -> Bitmap Index Scan on idx_logs_status (cost=0.00..4106.30 rows=376250
 width=0) (actual time=11.616..11.618 rows=374020.00 loops=1)
 Index Cond: (status = 'active'::text)
 Index Searches: 1
 Buffers: shared hit=1 read=317
 -> Parallel Hash (cost=6344.51..6344.51 rows=2 width=28) (actual
 time=28.123..28.128 rows=2.33 loops=3)
 Buckets: 1024 Batches: 1 Memory Usage: 72kB
 Buffers: shared hit=3246 read=3
 -> Hash Join (cost=8.45..6344.51 rows=2 width=28) (actual time=9.599..27.874
 rows=2.33 loops=3)
 Hash Cond: (t.user_id = u.id)
 Buffers: shared hit=3246 read=3
 -> Parallel Seq Scan on tasks t (cost=0.00..5789.17 rows=208333 width=22)
 (actual time=0.070..21.658 rows=166666.67 loops=3)
 Filter: (priority_range && '[20,80)'::int4range)"
 Buffers: shared hit=3235
 -> Hash (cost=8.44..8.44 rows=1 width=14) (actual time=0.072..0.074
 rows=1.00 loops=3)
 Buckets: 1024 Batches: 1 Memory Usage: 9kB
 Buffers: shared hit=11 read=3
 -> Index Scan using idx_users_username on users u (cost=0.42..8.44
 rows=1 width=14) (actual time=0.068..0.069 rows=1.00 loops=3)
 Index Cond: (username = 'user_142'::text)
 Index Searches: 3
 Buffers: shared hit=11 read=3
 Planning:
 Buffers: shared hit=68 read=2
 Planning Time: 0.528 ms
 Execution Time: 119.908 ms

Разница в реализации запроса: с индексами и без

1. Таблица users (условие: username = 'user_142')

Без индексов: Parallel Seq Scan — полное сканирование 100 000 строк, фильтрация 99 999 записей.

С индексами: Index Scan using idx_users_username — мгновенный поиск единственной строки через B-tree (фактическое время 0.069 мс).

Эффект: Индекс устранил чтение 99 999 лишних строк.

2. Таблица task_logs (условие: status = 'active')

Без индексов: Parallel Seq Scan — чтение всех 1 500 000 строк, фильтрация 376250 записей.

С индексами: Bitmap Index Scan на idx_logs_status → Bitmap Heap Scan — индекс быстро находит битовую карту страниц с активными записями, затем читает только нужные страницы.

Эффект: Индекс сократил количество читаемых страниц.

3. Таблица tasks (условие: priority_range && '[20,80)')

В обоих случаях: Parallel Seq Scan — индекс GiST не применяется.

Причина: Условие пересечения диапазонов возвращает ~166 667 строк из 500 000 (примерно треть). При такой селективности стоимость случайного чтения через GiST-дерево превышает стоимость параллельного последовательного сканирования. Оптимизатор выбирает Seq Scan.